

CS 3300 - Tools of the Trade #4

Demo Notes

Spring | Spring Boot | REST APIs

Big picture: We are creating a Java web application using Spring Boot. Spring Boot builds on Spring and simplifies setup through starter dependencies, auto-configuration, and an embedded web server. We will create a REST endpoint that receives an HTTP GET request and returns Java objects as JSON.

Demo 1: Create a Spring Boot Project

Goal: Bootstrap a Java web application using Spring Initializr.

1. Open the Command Palette (Ctrl+Shift+P) in VS Code.
2. Search for Spring Initializr and choose Spring Initializr: Create a Maven Project.
3. Select the Spring Boot version shown by the instructor.
4. Select Java and JDK 21.
5. Use group ID com.cs3300 and artifact ID demo_spring (or the values provided by the instructor).
6. Choose JAR packaging.
7. Add the Spring Web dependency.
8. Choose a project location and open the generated project in VS Code.

Key idea: Spring Initializr creates a Maven project for us. Maven still manages dependencies and builds the project; Spring Boot provides the application framework and simplifies its setup.

Demo 2: Inspect the Maven Project

9. Open pom.xml and locate the Spring Web starter.

```
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-web</artifactId>
</dependency>
```

Key idea: A Spring Boot starter is a convenient set of dependencies for a common application type. We asked for Spring Web instead of selecting the individual web libraries ourselves.

10. Open the generated main application class and locate SpringApplication.run(...).

Key idea: SpringApplication.run starts the Spring application and its IoC container. Spring can discover, create, and manage application components for us.

Demo 3: Create the Model

Goal: Create a Java class representing our data.

11. Create Student.java in the same package as the main application class.

```
public class Student {

    private int id;
```

```

private String name;

public Student(int id, String name) {
    this.id = id;
    this.name = name;
}

public int getId() { return id; }
public void setId(int id) { this.id = id; }
public String getName() { return name; }
public void setName(String name) { this.name = name; }
}

```

Key idea: Student is a regular Java data/domain object. There is nothing Spring-specific about this class.

Demo 4: Create a Service Managed by Spring

Goal: Move the operation for obtaining students into a service and let Spring manage it.

12. Create StudentService.java in the same package.

```

import org.springframework.stereotype.Service;
import java.util.Arrays;
import java.util.List;

@Service
public class StudentService {

    public List<Student> getAllStudents() {
        return Arrays.asList(
            new Student(1, "Michael"),
            new Student(2, "Alice")
        );
    }
}

```

Key idea: @Service tells Spring to create and manage a Spring bean for this class. We do not write new StudentService() in the controller.

Demo 5: Create the Controller and Use Dependency Injection

Goal: Inject StudentService into the controller.

13. Create StudentsController.java in the same package.

```

import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.RestController;
import java.util.List;

@RestController
public class StudentsController {

    private final StudentService studentService;

    public StudentsController(StudentService studentService) {

```

```

        this.studentService = studentService;
    }

    @GetMapping("/students")
    public List<Student> getAllStudents() {
        return studentService.getAllStudents();
    }
}

```

Key idea: StudentsController depends on StudentService, but it does not create it. Spring creates the service and supplies it through the constructor. This is constructor-based dependency injection.

Key idea: @RestController tells Spring this class handles web requests.

Key idea: @GetMapping("/students") maps an HTTP GET request for /students to this method. This is the same REST idea we used with Node.js and Express.

Key idea: IoC appears again: we do not manually call getAllStudents() when a request arrives. We declare the mapping, and Spring invokes the method when the matching request arrives.

Demo 6: Run the Application

14. Build the application with Maven:

```
mvn clean package
```

15. Look in target/ for the generated JAR. Run the actual JAR filename, for example:

```
java -jar target/demo_spring-0.0.1-SNAPSHOT.jar
```

Key idea: Maven is still our build tool. Spring Boot is the application framework and runtime setup.

Key idea: Watch the startup logs for the embedded Tomcat server and the port. We did not separately install Tomcat or deploy a WAR file.

Demo 7: Call the REST Endpoint

16. Open the endpoint in a browser:

```
http://localhost:8081/students
```

17. You should see JSON similar to:

```
[
  {"id": 1, "name": "Michael"},
  {"id": 2, "name": "Alice"}
]
```

Key idea: The method returns List<Student>, not a JSON string. Spring serializes the Java objects into JSON for the HTTP response.

Trace the full request

```

Browser
|
| GET /students

```

V
 Embedded Tomcat
 V
 StudentsController
 V
 StudentService
 V
 List<Student>
 V
 JSON response
 V
 Browser

Key idea: The browser sends an HTTP GET request. Spring routes it to the controller. The controller calls the injected service. The service returns Java objects, and Spring serializes them to JSON.

Demo 8: Configuration and Auto-Configuration

18. If port 8080 is already in use, open `src/main/resources/application.properties` and add:

```
server.port=8081
```

19. Open the application at `http://localhost:8081/students`.

Key idea: Spring Boot provides sensible defaults, but we can override them with configuration.

What You Should Be Able to Explain After the Demo

- What Maven does and how it differs from a framework.
- What Spring provides and what Spring Boot simplifies.
- What IoC means in a Spring application.
- What dependency injection is and why constructor injection is useful.
- What `@Service`, `@RestController`, and `@GetMapping` do.
- How an HTTP GET request reaches a Spring controller.
- How a Java `List<Student>` becomes JSON in the HTTP response.
- How Spring Boot uses starter dependencies, auto-configuration, and an embedded server.
- How this Java REST API relates to the Node.js/Express REST API from Tools of the Trade #3.